Pattern Recognition and Machine Learning

Chapter 4: Linear Models for Classification

Reading Notes

August 15, 2026

Table of Contents

1. Discriminant Functions
2. Probabilistic Generative Models
3. Probabilistic Discriminative Models
4. Laplace Approximation
5. Bayesian Logistic Regression

Linear Discriminant Functions

Two-Class Problem:

$$y(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0$$

- Decision boundary: $y(\mathbf{x}) = 0$
- $\mathbf{w}$ is orthogonal to decision surface
- w_0 determines position relative to origin

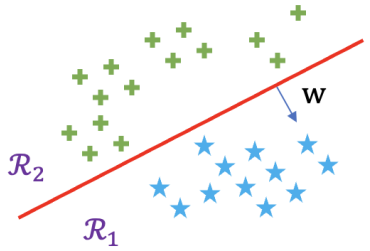
Multiple Classes (K-class discriminant):

- **Per-class** discriminant functions:

$$y_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x} + w_{k0}$$

- Matrix form (with $\mathbf{x}_0 = 1$):

$$\mathbf{y}(\mathbf{x}) = \widetilde{\mathbf{W}}^\top \widetilde{\mathbf{x}}$$



Linear Discriminant for Two Classes

Decision Rule

$k = \arg\max_j y_j(\mathbf{x}) \quad \Leftrightarrow \quad y_k(\mathbf{x}) > y_j(\mathbf{x}) \quad \forall j \neq k$

Least Squares for Classification

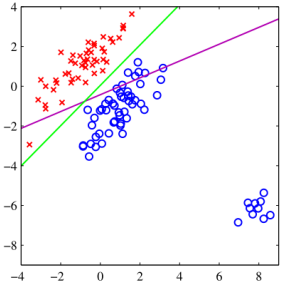
Loss Function:

$$E(\widetilde{\mathbf{W}}) = \frac{1}{2}\mathrm{Tr}\left[(\widetilde{\mathbf{X}}\widetilde{\mathbf{W}} - \mathbf{T})^\top(\widetilde{\mathbf{X}}\widetilde{\mathbf{W}} - \mathbf{T})\right]$$

Optimal Solution:

$$\widetilde{\mathbf{W}} = (\widetilde{\mathbf{X}}^\top\widetilde{\mathbf{X}})^{-1}\widetilde{\mathbf{X}}^\top\mathbf{T}$$

where $\widetilde{\mathbf{X}}$ is the design matrix with bias terms



Advantages:

- Closed-form solution
- Simple implementation
- Fast computation

Limitations:

- Sensitive to outliers
- Lack of probabilistic interpretation
- Poor performance on non-Gaussian data

Key Insight

Least squares minimizes squared error, which is inappropriate for discrete target variables.

Perceptron Algorithm

Perceptron Function:

$$y(\mathbf{x}) = f(\mathbf{w}^\top \mathbf{x})$$

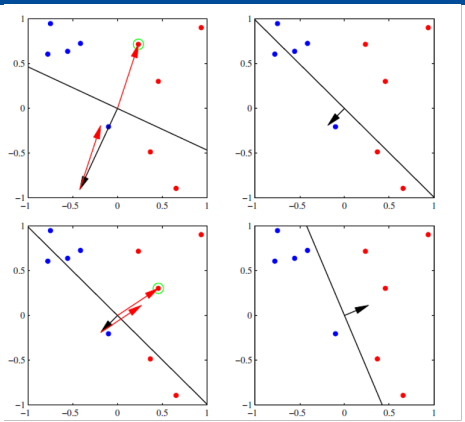
where $$f(a) = \begin{cases} +1 & \text{if } a \geq 0 \\ -1 & \text{if } a < 0 \end{cases}$$

Error Function:

$$E_P(\mathbf{w}) = -\sum_{n \in \mathcal{M}} \mathbf{w}^\top \mathbf{x}_n t_n$$

Update Rule:

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E_P(\mathbf{w}) = \mathbf{w}^{(\tau)} + \eta \mathbf{x}_n t_n$$



Perceptron Update Process

Convergence Theorem

If data is linearly separable, perceptron converges to exact solution in finite steps

Fisher's Linear Discriminant: Intuition

Core Idea: Find a linear projection that maximizes class separation

Key Insight:

- Project data onto 1D line
- Maximize between-class separation
- Minimize within-class variance
- Find optimal projection direction **w**

Projection:

$$y = \mathbf{w}^\top \mathbf{x} \quad (\|\mathbf{w}\| = 1)$$

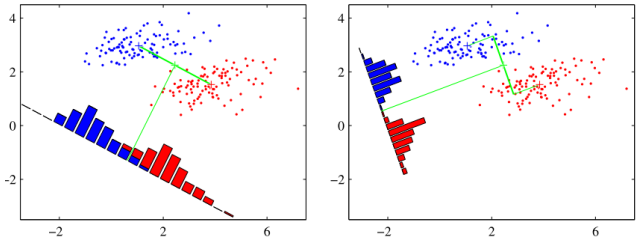


Figure: Fisher's LDA: Maximizing class separation

Geometric Interpretation

Find direction **w** that gives best class separation when data is projected onto this line

Fisher's Linear Discriminant: Mathematics

Within-Class Scatter Matrix:

$$\mathbf{S}_W = \sum_{k=1}^{K} \mathbf{S}_k = \sum_{k=1}^{K} \sum_{n \in \mathcal{C}_k} (\mathbf{x}_n - \mathbf{m}_k)(\mathbf{x}_n - \mathbf{m}_k)^\top \quad \text{where } \mathbf{m}_k = \frac{1}{N_k} \sum_{n \in \mathcal{C}_k} \mathbf{x}_n$$

Between-Class Scatter Matrix:

$$\mathbf{S}_B = \sum_{k=1}^{K} N_k (\mathbf{m}_k - \mathbf{m})(\mathbf{m}_k - \mathbf{m})^\top \quad \text{where } \mathbf{m} = \frac{1}{N} \sum_{n=1}^{N} \mathbf{x}_n$$

Fisher Criterion:

$$J(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}$$

Optimization Goal

Maximize between-class separation relative to within-class variance

Fisher's Linear Discriminant: Derivation

Optimization Problem:

$$\max_{\mathbf{w}} J(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}$$

Two-Class Simplification:

$$\mathbf{S}_B = (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^\top$$

Take Derivative:

$$\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} = 0 \quad \Rightarrow \quad (\mathbf{w}^\top \mathbf{S}_W \mathbf{w})\mathbf{S}_B \mathbf{w} = (\mathbf{w}^\top \mathbf{S}_B \mathbf{w})\mathbf{S}_W \mathbf{w}$$

$$\begin{aligned} \mathbf{S}_B \mathbf{w} &= \lambda \mathbf{S}_W \mathbf{w} \\ (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^\top \mathbf{w} &= \lambda \mathbf{S}_W \mathbf{w} \\ k(\mathbf{m}_1 - \mathbf{m}_2) &= \lambda \mathbf{S}_W \mathbf{w} \\ \mathbf{w} &\propto \mathbf{S}_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2) \end{aligned}$$

Fisher's Linear Discriminant: Classification

After Projection: Data is projected to 1D space

$$y = \mathbf{w}^\top \mathbf{x}, \quad \text{where } \mathbf{w} \propto \mathbf{S}_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$$

Projected Class Means:

$$\tilde{m}_1 = \mathbf{w}^\top \mathbf{m}_1, \quad \tilde{m}_2 = \mathbf{w}^\top \mathbf{m}_2$$

Threshold Choices:

- **Midpoint**: $y_0 = \frac{\tilde{m}_1 + \tilde{m}_2}{2}$
- **Sample-weighted**: $y_0 = \frac{N_1 \tilde{m}_1 + N_2 \tilde{m}_2}{N_1 + N_2}$
- **Variance-weighted**: $y_0 = \frac{\tilde{s}_2^2 \tilde{m}_1 + \tilde{s}_1^2 \tilde{m}_2}{\tilde{s}_1^2 + \tilde{s}_2^2}$

Threshold Selection Guide

- **Midpoint**: Equal weighting of both classes
- **Sample-weighted**: Weighted by class sample sizes
- **Variance-weighted**: Weighted by class variances

Generative Approach

Core Idea: Model class distributions, then use Bayes' theorem

Bayes' Theorem (Two-Class):

$$p(\mathcal{C}_1|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1) + p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)}$$

Sigmoid Representation:

$$p(\mathcal{C}_1|\mathbf{x}) = \sigma(a) = \frac{1}{1+\exp(-a)} \quad \text{where } a = \ln \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)}$$

$$a = \ln\left(\frac{\sigma(a)}{1-\sigma(a)}\right) = \ln\left(\frac{p(\mathcal{C}_1|\mathbf{x})}{p(\mathcal{C}_2|\mathbf{x})}\right)$$

Key Advantage

Can generate synthetic data by sampling from $p(\mathbf{x}|\mathcal{C}_k)$

Continuous Inputs: Gaussian Discriminant Analysis

Model Assumption: Class-conditional distributions are Gaussian with shared covariance

$$p(\mathbf{x}|\mathcal{C}_k) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma})$$

$$\begin{aligned} a &= \ln \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)} = \ln \frac{\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_1, \boldsymbol{\Sigma})p(\mathcal{C}_1)}{\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_2, \boldsymbol{\Sigma})p(\mathcal{C}_2)} \\ &= \mathbf{x}^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) - \frac{1}{2}\boldsymbol{\mu}_1^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_2^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 + \ln \frac{p(\mathcal{C}_1)}{p(\mathcal{C}_2)} \\ \mathbf{w} &= \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) \\ w_0 &= -\frac{1}{2}\boldsymbol{\mu}_1^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_2^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 + \ln \frac{p(\mathcal{C}_1)}{p(\mathcal{C}_2)} \end{aligned}$$

Final Result: $p(\mathcal{C}_1|\mathbf{x}) = \sigma(a) = \sigma(\mathbf{w}^\top \mathbf{x} + w_0)$

Key Insight

Shared covariance structure $\Rightarrow$ Linear decision boundary

Maximum Likelihood Estimation

Likelihood Function:

$$p(\mathcal{D}|\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}) = \prod_{k=1}^{2} \prod_{n \in \mathcal{C}_k} \mathcal{N}(\mathbf{x}_n|\boldsymbol{\mu}_k, \boldsymbol{\Sigma})$$

Maximizing with respect to parameters:

- **Class Means**:

$$\frac{\partial}{\partial \boldsymbol{\mu}_k} \ln p(\mathcal{D}|\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}) = 0 \quad \Rightarrow \quad \boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n \in \mathcal{C}_k} \mathbf{x}_n$$

- **Shared Covariance**:

$$\frac{\partial}{\partial \boldsymbol{\Sigma}} \ln p(\mathcal{D}|\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}) = 0 \quad \Rightarrow \quad \boldsymbol{\Sigma} = \frac{1}{N} \sum_{k=1}^{2} \sum_{n \in \mathcal{C}_k} (\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^\top$$

GDA Limitations

Requires shared covariance and Gaussian assumptions, which is sensitive to outliers.

Discrete Inputs: Naive Bayes Classifier

Model Assumption: Features are conditionally independent given class

$$p(\mathbf{x}|\mathcal{C}_k) = \prod_{i=1}^{D} \mu_{ki}^{x_i}(1-\mu_{ki})^{1-x_i} \qquad \mu_{ki} = p(x_i = 1|\mathcal{C}_k)$$

Posterior Derivation:

$$\begin{aligned} p(\mathcal{C}_1|\mathbf{x}) &= \sigma\left(\ln \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)}\right) \\ &= \sigma\left(\sum_{i=1}^{D} x_i \ln \frac{\mu_{1i}}{\mu_{2i}} + \sum_{i=1}^{D} \ln \frac{1-\mu_{1i}}{1-\mu_{2i}} + \ln \frac{p(\mathcal{C}_1)}{p(\mathcal{C}_2)}\right) \end{aligned}$$

Final Result: $p(\mathcal{C}_1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + w_0)$

Key Insight

Conditional independence assumption $\Rightarrow$ Linear decision boundaries despite discrete inputs

Discriminative vs Generative Approach

Generative Models:

$$p(\mathcal{C}_k|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{p(\mathbf{x})}$$

Model class-conditional densities

- Learn $p(\mathbf{x}|\mathcal{C}_k)$ and $p(\mathcal{C}_k)$
- Can generate synthetic data
- More parameters needed
- Examples: LDA, QDA, Naive Bayes

Discriminative Models:

$p(\mathcal{C}_k|\mathbf{x})$ modeled directly
(no density modeling)

- Directly learn $p(\mathcal{C}_k|\mathbf{x})$
- Cannot generate data
- Fewer parameters
- Better performance often
- Examples: Logistic Regression

Key Insight

Discriminative models focus only on the decision boundary, ignoring data distribution details

Logistic Regression: Motivation

From Generative to Discriminative:
For two classes with Gaussian distributions having **shared covariance**:

$$p(\mathcal{C}_1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + w_0)$$
$$\mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$$
$$w_0 = -\frac{1}{2}\boldsymbol{\mu}_1^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_2^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 + \ln \frac{p(\mathcal{C}_1)}{p(\mathcal{C}_2)}$$

Logistic Regression Approach:

- **Assume**: $p(\mathcal{C}_1|\mathbf{x}) = \sigma(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}))$
- **Estimate**: $\mathbf{w}$ directly from data (MLE/MAP)
- **No need**: To assume Gaussian distributions
- **Advantage**: More flexible, fewer assumptions

Parameter Count Comparison

For D-dimensional features: Generative needs $O(D^2)$ parameters (covariance), Discriminative needs only $O(D)$

Logistic Regression: Model Specification

Binary Classification:

$$p(\mathcal{C}_1|\boldsymbol{\phi}) = \sigma(\mathbf{w}^\top \boldsymbol{\phi}) = \frac{1}{1+\exp(-\mathbf{w}^\top \boldsymbol{\phi})}$$
$$p(\mathcal{C}_2|\boldsymbol{\phi}) = 1 - \sigma(\mathbf{w}^\top \boldsymbol{\phi}) = \sigma(-\mathbf{w}^\top \boldsymbol{\phi})$$

Basis Functions $\boldsymbol{\phi}(\mathbf{x})$:

- Linear: $\boldsymbol{\phi}(\mathbf{x}) = [1, x_1, \ldots, x_D]^\top$
- Polynomial: $\boldsymbol{\phi}(\mathbf{x}) = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2, \ldots]^\top$
- Nonlinear: $\boldsymbol{\phi}(\mathbf{x}) = [1, \phi_1(\mathbf{x}), \ldots, \phi_M(\mathbf{x})]^\top$

Log-Odds Representation:

$$\ln \frac{p(\mathcal{C}_1|\boldsymbol{\phi})}{p(\mathcal{C}_2|\boldsymbol{\phi})} = \mathbf{w}^\top \boldsymbol{\phi}$$

Model Properties

- Decision boundary: $\mathbf{w}^\top \boldsymbol{\phi} = 0$ (linear in feature space)

Maximum Likelihood Estimation

Dataset: $\{\phi_n, t_n\}_{n=1}^N$, where $t_n \in \{0, 1\}$

Likelihood Function:

$$\begin{aligned} p(\mathbf{t}|\mathbf{w}) &= \prod_{n=1}^{N} p(\mathcal{C}_1|\phi_n)^{t_n}[1 - p(\mathcal{C}_1|\phi_n)]^{1-t_n} \\ &= \prod_{n=1}^{N} y_n^{t_n}(1 - y_n)^{1-t_n} \end{aligned}$$

where $y_n = \sigma(\mathbf{w}^\top \phi_n) = p(\mathcal{C}_1|\phi_n)$

Cross-Entropy Error Function:

$$\begin{aligned} E(\mathbf{w}) &= -\ln p(\mathbf{t}|\mathbf{w}) \\ &= -\sum_{n=1}^{N}[t_n \ln y_n + (1 - t_n)\ln(1 - y_n)] \end{aligned}$$

Gradient and Hessian

Gradient Calculation:

$$\nabla E(\mathbf{w}) = \sum_{n=1}^{N} \frac{\partial E_n}{\partial y_n} \frac{\partial y_n}{\partial \mathbf{w}}$$

$$\frac{\partial E_n}{\partial y_n} = \frac{y_n - t_n}{y_n(1 - y_n)}$$

$$\frac{\partial y_n}{\partial \mathbf{w}} = y_n(1 - y_n)\phi_n$$

$$\Rightarrow \nabla E(\mathbf{w}) = \sum_{n=1}^{N} (y_n - t_n)\phi_n$$

Hessian Matrix (Second Derivatives):

$$\mathbf{H} = \nabla\nabla E(\mathbf{w}) = \sum_{n=1}^{N} \nabla[(y_n - t_n)\phi_n]$$

Iterative Reweighted Least Squares (IRLS)

Newton-Raphson Update:

$$\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} - \mathbf{H}^{-1}\nabla E(\mathbf{w})$$

IRLS Derivation:

$$\begin{aligned}\mathbf{w}^{\text{new}} &= \mathbf{w}^{\text{old}} - (\mathbf{\Phi}^\top \mathbf{R}\mathbf{\Phi})^{-1}\mathbf{\Phi}^\top(\mathbf{y}-\mathbf{t}) \\ &= (\mathbf{\Phi}^\top \mathbf{R}\mathbf{\Phi})^{-1}[\mathbf{\Phi}^\top \mathbf{R}\mathbf{\Phi}\mathbf{w}^{\text{old}} - \mathbf{\Phi}^\top(\mathbf{y}-\mathbf{t})] \\ &= (\mathbf{\Phi}^\top \mathbf{R}\mathbf{\Phi})^{-1}\mathbf{\Phi}^\top \mathbf{R}\mathbf{z}\end{aligned}$$

where $\mathbf{z} = \mathbf{\Phi}\mathbf{w}^{\text{old}} - \mathbf{R}^{-1}(\mathbf{y}-\mathbf{t})$ is the "working response"

Algorithm Steps:

1. Initialize $\mathbf{w}^{(0)}$
2. Compute $\mathbf{y} = \sigma(\mathbf{\Phi}\mathbf{w})$
3. Compute $\mathbf{R} = \text{diag}\{y_n(1-y_n)\}$
4. Compute $\mathbf{z} = \mathbf{\Phi}\mathbf{w} - \mathbf{R}^{-1}(\mathbf{y}-\mathbf{t})$

Properties:

- Quadratic convergence near optimum
- Each iteration solves weighted least squares
- Weights $y_n(1-y_n)$ give less weight to

Multi-class Logistic Regression

Softmax Function:

$$p(\mathcal{C}_k|\phi) = \frac{\exp(a_k)}{\sum_{j=1}^{K} \exp(a_j)}$$

where $a_k = \mathbf{w}_k^\top \phi$ are the **activations**

Properties:

- $0 < p(\mathcal{C}_k|\phi) < 1$ for all k
- $\sum_{k=1}^{K} p(\mathcal{C}_k|\phi) = 1$
- Invariant to adding constant to all a_k
- Reduces to sigmoid when $K = 2$

Likelihood Function:

$$p(\mathbf{T}|\mathbf{w}_1, \ldots, \mathbf{w}_K) = \prod_{n=1}^{N} \prod_{k=1}^{K} p(\mathcal{C}_k|\phi_n)^{t_{nk}}$$

where $t_{nk} = 1$ if $\mathbf{x}_n$ belongs to class k, 0 otherwise

Cross-Entropy Error:

$$N \quad K$$

Multi-class Gradient and Optimization

Gradient Calculation:

$$\frac{\partial E}{\partial \mathbf{w}_j} = \sum_{n=1}^{N} \frac{\partial E_n}{\partial a_{nj}} \frac{\partial a_{nj}}{\partial \mathbf{w}_j}$$

$$\frac{\partial E_n}{\partial a_{nj}} = \frac{\partial}{\partial a_{nj}} \left[-\sum_k t_{nk} \ln \frac{\exp(a_{nk})}{\sum_l \exp(a_{nl})} \right]$$

$$= p(\mathcal{C}_j|\phi_n) - t_{nj}$$

$$\Rightarrow \frac{\partial E}{\partial \mathbf{w}_j} = \sum_{n=1}^{N} (p(\mathcal{C}_j|\phi_n) - t_{nj})\phi_n$$

Parameter Identifiability Issue:

- Adding constant vector to all $\mathbf{w}_k$ doesn't change probabilities
- Solution: Set $\mathbf{w}_K = \mathbf{0}$ (one weight vector as reference)
- Or add regularization (e.g., $\lambda \sum_k \|\mathbf{w}_k\|^2$)

Regularization and Bayesian Perspective

Overfitting Problem: Logistic regression can overfit with many features

Regularized Error Function:

$$E_{\text{reg}}(\mathbf{w}) = E(\mathbf{w}) + \frac{\lambda}{2}\|\mathbf{w}\|^2$$

where $\lambda > 0$ is regularization coefficient

Bayesian Interpretation:

- Regularization corresponds to Gaussian prior:

$$p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \lambda^{-1}\mathbf{I})$$

- MAP estimate: $\mathbf{w}_{\text{MAP}} = \arg\max_{\mathbf{w}} p(\mathbf{t}|\mathbf{w})p(\mathbf{w})$
- Equivalent to minimizing $E_{\text{reg}}(\mathbf{w})$

Hyperparameter Selection:

- Cross-validation to choose λ
- Bayesian approach: Marginalize over $\mathbf{w}$ or use empirical Bayes
- Information criteria: AIC, BIC for model selection

Extensions and Practical Considerations

Feature Engineering:

- Polynomial features for nonlinear boundaries
- Interaction terms
- Kernel logistic regression
- Feature scaling important for gradient descent

Numerical Issues:

- Sigmoid saturation: $\sigma(a) \approx 0$ or 1 for large $|a|$
- Log-likelihood can underflow
- Use log-sum-exp trick for stability

Implementation Tips:

- Initialize weights near zero
- Monitor training/test error
- Early stopping for regularization
- Mini-batch updates for large datasets

Advanced Variants:

- Multinomial logistic regression
- Hierarchical logistic regression
- Bayesian logistic regression
- Regularized logistic regression (LASSO, Elastic Net)

When to Use Logistic Regression

Laplace Approximation Method

Objective: Approximate complex posterior with Gaussian

Procedure:

1. Find mode $\mathbf{w}_{\text{MAP}}$ of posterior
2. Compute Hessian: $\mathbf{A} = -\nabla\nabla \ln p(\mathbf{w}|\mathcal{D})$
3. Gaussian approximation: $q(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{w}_{\text{MAP}}, \mathbf{A}^{-1})$

Mathematical Basis:

$$\ln p(\mathbf{w}|\mathcal{D}) \approx \ln p(\mathbf{w}_{\text{MAP}}|\mathcal{D}) - \frac{1}{2}(\mathbf{w} - \mathbf{w}_{\text{MAP}})^{\top}\mathbf{A}(\mathbf{w} - \mathbf{w}_{\text{MAP}})$$

Applicability

Works well for unimodal, approximately Gaussian posteriors

Model Evidence and BIC

Model Evidence:

$$p(\mathcal{D}) = \int p(\mathcal{D}|\mathbf{w})p(\mathbf{w})d\mathbf{w}$$

Laplace Approximation:

$$\ln p(\mathcal{D}) \approx \ln p(\mathcal{D}|\mathbf{w}_{\text{MAP}}) + \ln p(\mathbf{w}_{\text{MAP}}) + \frac{M}{2}\ln(2\pi) - \frac{1}{2}\ln|\mathbf{A}|$$

BIC:

$$\ln p(\mathcal{D}) \approx \ln p(\mathcal{D}|\mathbf{w}_{\text{MAP}}) - \frac{M}{2}\ln N$$

AIC:

$$\ln p(\mathcal{D}) \approx \ln p(\mathcal{D}|\mathbf{w}_{\text{MAP}}) - M$$

Model Selection

Trade-off between model fit and complexity

Bayesian Framework

Predictive Distribution:

$$p(\mathcal{C}_1|\mathbf{t}) = \int \sigma(\mathbf{w}^\top \phi) p(\mathbf{w}|\mathbf{t}) d\mathbf{w}$$

Computational Challenges

- Posterior $p(\mathbf{w}|\mathbf{t})$ is non-Gaussian
- Predictive distribution is intractable
- Requires approximation methods

Approximation Methods:

- Laplace approximation
- Variational inference (Chapter 10)
- MCMC sampling (Chapter 11)

Laplace Approximation Application

Posterior Approximation:

$$p(\mathbf{w}|\mathbf{t}) \approx \mathcal{N}(\mathbf{w}|\mathbf{w}_{\text{MAP}}, \mathbf{S}_N)$$

where $\mathbf{S}_N^{-1} = \mathbf{S}_0^{-1} + \sum_{n=1}^{N} y_n(1-y_n)\boldsymbol{\phi}_n\boldsymbol{\phi}_n^\top$

Predictive Distribution:

$$p(\mathcal{C}_1|\boldsymbol{\phi}, \mathbf{t}) \approx \sigma\left(\kappa(\sigma_a^2)\mu_a\right)$$

where

$$\mu_a = \mathbf{w}_{\text{MAP}}^\top \boldsymbol{\phi}$$
$$\sigma_a^2 = \boldsymbol{\phi}^\top \mathbf{S}_N \boldsymbol{\phi}$$
$$\kappa(\sigma^2) = (1 + \pi\sigma^2/8)^{-1/2}$$

Practical Value

Good balance between computational efficiency and accuracy

Chapter Summary

- **Discriminant Functions**: Direct classification approaches
  - Fisher's LDA, perceptron, least squares
- **Generative Models**: Model class conditionals + priors
  - Gaussian assumptions, naive Bayes
  - Can generate synthetic data
- **Discriminative Models**: Direct posterior modeling
  - Logistic regression, IRLS optimization
  - Generally better performance
- **Bayesian Methods**: Full probabilistic treatment
  - Laplace approximation for tractability
  - Uncertainty quantification in predictions

Key Takeaways

- Different assumptions lead to complementary approaches
- Bayesian methods naturally handle uncertainty
- Approximate inference enables practical Bayesian computation